connectionist models are not reliable for certain tasks

Example

In [1]:
```
"""Neural net to take the product of three numbers
After https://gist.github.com/qpwo/34983724ed9333225974191798bb672d
"""
from tensorflow.keras import models, layers
import numpy as np

np.set_printoptions(formatter={'float': lambda x: "{0:0.3f}".format(x)})

x = np.random.random((100, 3))
y = np.prod(x, axis=1).reshape(100, 1)

model = models.Sequential([
    layers.Dense(3, activation='relu', input_shape=(3,)),
    layers.Dense(3, activation='relu'),
    layers.Dense(3, activation='relu'),
    layers.Dense(1, activation='relu')
])
model.compile(loss='mean_absolute_error', optimizer='adam')
model.fit(x, y, epochs=1000)

# small test, to see quality of results

x2 = np.random.random((5, 3))
y2 = np.prod(x2, axis=1).reshape(5,1)
p2 = model.predict(x2)

print("x:\n", x2)
print("y:\n", y2)
print("predicted:\n", p2)
```

```
C:\Users\yathi\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:106: UserWarning: Do no
t pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer usi
ng an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.
11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-Dir
ectML plugin.
Epoch 1/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - loss: 0.1427
Epoch 2/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 3/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.1427
Epoch 4/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 5/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 6/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 7/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 8/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 9/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 10/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 11/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 12/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 13/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 14/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 15/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 16/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 17/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 18/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 19/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 20/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 21/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 22/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 23/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 24/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 25/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 26/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 27/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 28/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 29/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 30/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 31/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 32/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 33/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 34/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 35/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 36/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 37/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 38/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 39/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 40/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 41/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 42/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 43/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 44/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 45/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 46/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 47/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 48/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 49/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 50/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.1427
Epoch 51/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 52/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 53/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 54/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 55/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 56/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 57/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 58/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 59/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 60/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 61/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 62/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 63/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 64/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 65/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.1427
Epoch 66/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 67/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 68/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 69/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 70/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 71/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 72/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 73/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 74/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 75/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 76/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 77/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 78/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 79/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 80/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 81/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 82/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 83/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 84/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 85/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 86/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 87/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 88/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 89/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 90/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 91/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 92/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 93/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 94/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 95/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 96/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 97/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 98/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.1427
Epoch 99/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 100/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 101/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 102/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 103/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 104/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 105/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 106/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 107/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 108/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 109/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 110/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 111/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 112/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.1427
Epoch 113/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 114/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 115/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 116/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 117/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 118/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 119/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 120/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 121/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 122/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 123/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 124/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 125/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 126/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 127/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 128/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 129/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 130/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 131/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 132/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 133/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 134/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 135/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 136/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 137/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 138/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 139/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 140/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 141/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 142/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 143/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 144/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 145/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 146/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 147/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 148/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 149/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 150/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 151/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 152/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 153/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 154/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 155/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.1427
Epoch 156/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 157/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 158/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 159/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.1427
Epoch 160/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 161/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 162/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 163/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 164/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 165/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 166/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 167/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 168/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 169/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 170/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 171/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 172/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 173/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 174/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 175/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 176/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 177/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 178/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 179/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 180/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 181/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 182/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 183/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 184/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 185/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 186/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 187/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 188/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 189/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 190/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 191/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 192/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 193/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.1427
Epoch 194/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.1427
Epoch 195/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 196/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 197/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 198/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 199/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 200/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 201/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 202/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 203/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 204/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 205/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 206/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 207/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 208/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 209/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 210/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 211/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 212/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 213/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 214/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 215/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 216/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 217/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 218/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 219/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 220/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 221/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 222/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 223/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 224/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 225/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 226/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 227/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 228/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 229/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 230/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 231/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 232/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 233/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 234/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 235/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 236/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 237/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 238/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 239/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 240/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 241/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 242/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 243/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 244/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 245/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 246/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 247/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 248/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 249/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 250/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 251/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 252/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 253/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 254/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 255/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 256/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 257/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 258/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 259/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 260/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 261/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 262/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.1427
Epoch 263/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 264/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 265/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 266/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 267/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 268/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 269/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 270/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 271/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 272/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 273/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 274/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 275/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 276/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 277/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 278/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 279/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 280/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 281/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 282/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 283/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 284/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 285/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 286/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 287/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 288/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 289/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 290/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 291/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 292/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 293/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 294/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 295/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 296/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 297/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 298/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 299/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 300/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 301/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 302/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 303/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 304/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 305/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 306/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 307/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 308/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 309/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 310/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 311/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 312/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 313/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 314/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 315/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.1427
Epoch 316/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 317/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 318/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 319/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 320/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 321/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 322/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 323/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 324/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 325/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 326/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 327/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 328/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 329/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 330/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 331/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 332/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 333/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 334/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 335/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 336/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 337/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 338/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 339/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 340/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 341/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 342/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 343/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 344/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 345/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.1427
Epoch 346/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 347/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.1427
Epoch 348/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 349/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 350/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 351/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 352/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 353/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 354/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 355/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 356/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 357/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 358/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 359/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 360/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 361/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 362/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 363/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 364/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 365/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 366/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 367/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 368/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 369/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 370/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 371/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 372/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 373/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 374/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 375/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 376/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 377/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 378/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 379/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 380/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 381/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 382/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 383/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 384/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 385/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 386/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 387/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 388/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 389/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 390/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 391/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 392/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 393/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 394/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 395/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 396/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 397/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 398/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 399/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 400/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 401/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 402/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 403/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 404/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 405/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 406/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 407/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 408/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 409/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 410/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 411/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 412/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 413/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 414/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 415/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 416/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 417/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 418/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 419/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 420/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 421/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 422/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 423/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 424/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 425/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 426/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 427/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 428/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 429/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 430/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 431/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 432/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 433/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 434/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 435/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 436/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 437/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 438/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 439/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 440/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 441/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 442/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 443/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 444/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 445/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.1427
Epoch 446/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 447/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 448/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 449/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 450/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 451/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 452/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 453/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 454/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 455/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 456/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 457/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 458/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 459/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 460/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 461/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 462/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 463/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 464/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 465/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 466/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 467/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 468/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 469/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 470/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 471/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 472/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 473/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 474/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 475/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 476/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 477/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 478/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 479/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 480/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 481/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 482/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 483/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 484/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 485/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 486/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 487/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 488/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 489/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 490/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 491/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 492/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 493/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 494/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 495/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 496/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 497/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 498/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 499/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 500/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 501/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 502/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 503/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 504/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 505/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 506/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 507/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 508/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 509/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 510/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 511/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 512/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 513/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 514/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 515/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 516/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 517/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 518/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 519/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 520/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 521/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 522/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 523/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 524/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 525/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 526/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 527/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 528/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 529/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 530/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 531/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 532/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 533/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 534/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 535/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 536/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 537/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 538/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 539/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 540/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 541/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 542/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 543/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 544/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 545/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 546/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 547/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 548/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 549/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 550/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 551/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 552/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 553/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 554/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 555/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 556/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 557/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 558/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 559/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 560/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 561/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 562/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 563/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 564/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 565/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 566/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 567/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 568/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 569/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 570/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 571/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 572/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 573/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 574/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 575/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 576/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 577/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 578/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 579/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 580/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 581/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 582/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 583/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 584/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 585/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 586/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 587/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 588/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 589/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 590/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 591/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 592/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 593/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 594/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 595/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 596/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 597/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 598/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 599/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 600/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 601/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 602/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 603/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 604/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 605/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 606/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 607/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 608/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 609/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 610/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 611/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 612/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 613/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 614/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 615/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 616/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 617/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 618/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 619/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 620/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 621/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 622/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 623/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 624/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 625/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 626/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 627/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 628/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 629/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 630/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 631/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 632/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 633/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 634/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 635/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 636/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 637/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 638/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.1427
Epoch 639/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 640/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 641/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 642/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 643/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 644/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 645/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 646/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 647/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 648/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 649/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 650/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 651/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 652/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 653/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 654/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 655/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 656/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 657/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 658/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 659/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 660/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 661/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 662/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 663/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 664/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 665/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 666/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 667/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 668/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 669/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 670/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 671/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 672/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 673/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 674/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 675/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 676/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 677/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 678/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 679/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 680/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 681/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 682/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 683/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 684/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 685/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 686/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 687/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 688/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 689/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 690/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 691/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 692/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 693/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 694/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 695/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 696/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 697/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 698/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 699/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 700/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 701/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 702/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 703/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 704/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 705/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 706/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 707/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 708/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 709/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 710/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 711/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 712/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 713/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 714/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 715/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 716/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 717/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 718/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 719/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 720/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 721/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 722/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 723/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 724/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 725/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 726/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 727/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 728/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 729/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 730/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 731/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 732/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 733/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 734/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 735/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 736/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 737/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 738/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 739/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 740/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 741/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 742/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 743/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 744/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 745/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 746/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 747/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 748/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 749/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 750/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 751/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 752/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 753/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 754/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 755/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 756/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 757/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 758/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 759/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 760/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 761/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 762/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 763/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 764/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 765/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 766/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 767/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 768/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 769/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 770/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 771/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 772/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 773/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 774/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 775/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 776/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 777/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 778/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 779/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 780/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 781/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 782/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 783/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 784/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 785/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 786/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 787/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 788/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 789/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 790/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 791/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 792/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 793/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 794/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 795/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 796/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 797/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 798/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 799/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 800/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 801/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 802/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 803/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 804/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 805/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 806/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 807/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 808/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 809/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 810/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 811/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 812/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 813/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 814/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 815/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 816/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 817/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 818/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 819/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 820/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 821/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 822/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 823/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 824/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 825/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 826/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 827/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 828/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 829/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 830/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 831/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 832/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 833/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 834/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 835/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 836/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 837/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 838/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 839/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 840/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 841/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 842/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 843/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 844/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 845/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 846/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 847/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 848/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 849/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 850/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 851/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 852/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 853/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 854/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 855/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 856/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 857/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 858/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 859/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 860/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 861/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 862/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 863/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 864/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 865/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 866/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 867/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 868/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 869/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 870/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 871/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 872/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 873/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 874/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 875/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 876/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 877/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 878/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 879/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 880/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 881/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 882/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 883/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 884/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 885/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 886/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 887/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 888/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 889/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 890/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 891/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 892/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 893/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 894/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 895/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 896/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 897/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 898/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 899/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 900/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 901/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 902/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 903/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 904/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 905/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 906/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 907/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 908/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 909/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 910/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 911/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 912/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 913/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 914/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 915/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 916/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 917/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 918/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 919/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 920/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 921/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 922/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 923/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 924/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 925/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 926/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 927/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 928/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 929/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 930/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 931/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 932/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 933/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 934/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 935/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 936/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 937/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 938/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 939/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 940/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 941/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 942/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 943/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 944/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 945/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 946/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 947/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 948/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 949/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 950/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 951/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 952/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 953/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 954/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 955/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 956/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 957/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 958/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 959/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 960/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 961/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 962/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 963/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 964/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 965/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 966/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 967/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 968/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 969/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 970/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 971/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 972/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 973/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 974/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 975/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 976/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 977/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 978/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 979/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 980/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 981/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 982/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 983/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 984/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
Epoch 985/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.1427
Epoch 986/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 987/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 988/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 989/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.1427
Epoch 990/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 991/1000
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 992/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 993/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 994/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 995/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 996/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.1427
Epoch 997/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 998/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 999/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.1427
Epoch 1000/1000
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.1427
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 76ms/step
x:
 [[0.082 0.012 0.312]
 [0.932 0.891 0.639]
 [0.614 0.340 0.050]
 [0.998 0.042 0.937]
 [0.521 0.343 0.248]]
y:
 [[0.000]
 [0.530]
 [0.010]
 [0.039]
 [0.044]]
predicted:
 [[0.000]
 [0.000]
 [0.000]
 [0.000]
 [0.000]]
```

In [2]:

```
from nltk import *
from nltk.sem.drt import DrtParser
from nltk.sem import logic
logic._counter._value = 0
```

In [3]:

```
# from http://www.nltk.org/howto/inference.html
from nltk.sem import Expression
p1 = Expression.fromstring('man(socrates)')
p2 = Expression.fromstring('all x.(man(x) -> mortal(x))')
c  = Expression.fromstring('mortal(socrates)')
Prover9().prove(c, [p1,p2])
```

Out[3]:

```
True
```

In [4]:

```
import os
for k, v in os.environ.items():
    if "PROVER" in k.upper():
        print(k, "=", v)
```

```
PROVER9 = C:\Program Files (x86)\Prover9-Mace4\bin-win32
```

In [5]:

```
TableauProver().prove(c, [p1, p2])
```

Out[5]: True

In [6]:
```
ResolutionProver().prove(c, [p1, p2], verbose=True)
```

```
[1] {-mortal(socrates)}     A
[2] {man(socrates)}         A
[3] {-man(z2), mortal(z2)}  A
[4] {-man(socrates)}        (1, 3)
[5] {mortal(socrates)}      (2, 3)
[6] {}                      (1, 5)
```

Out[6]: True

In [7]:
```
#from rdf2vec.converters import create_kg
import pandas as pd

# get the zoo dataset from https://www.kaggle.com/uciml/zoo-animal-classification#
zoo = pd.read_csv('zoo.csv')
```

In [8]:
```
cols = list(zoo.columns)
cols
```

Out[8]:
```
['animal_name',
 'hair',
 'feathers',
 'eggs',
 'milk',
 'airborne',
 'aquatic',
 'predator',
 'toothed',
 'backbone',
 'breathes',
 'venomous',
 'fins',
 'legs',
 'tail',
 'domestic',
 'catsize',
 'class_type']
```

In [9]:
```
zoo.columns[zoo.nunique() == 2]
```

Out[9]:
```
Index(['hair', 'feathers', 'eggs', 'milk', 'airborne', 'aquatic', 'predator',
       'toothed', 'backbone', 'breathes', 'venomous', 'fins', 'tail',
       'domestic', 'catsize'],
      dtype='str')
```

In [10]:
```
binary_cols = zoo.columns[zoo.nunique() == 2]
binary_cols
```

Out[10]:
```
Index(['hair', 'feathers', 'eggs', 'milk', 'airborne', 'aquatic', 'predator',
       'toothed', 'backbone', 'breathes', 'venomous', 'fins', 'tail',
       'domestic', 'catsize'],
      dtype='str')
```

In [11]:
```
for col in binary_cols:
    zoo[col] = zoo[col].astype(bool)
```

In [12]:
```
zoo.dtypes
```

Out[12]:
```
animal_name      str
hair            bool
feathers        bool
eggs            bool
milk            bool
airborne        bool
aquatic         bool
predator        bool
toothed         bool
backbone        bool
breathes        bool
venomous        bool
fins            bool
legs           int64
tail            bool
domestic        bool
catsize         bool
class_type     int64
dtype: object
```

In [13]:
```
len(zoo)
```

Out[13]:
```
101
```

In [14]:
```
zoo.head()
```

Out[14]:

	animal_name	hair	feathers	eggs	milk	airborne	aquatic	predator	toothed	backbone	breathes
0	aardvark	True	False	False	True	False	False	True	True	True	True
1	antelope	True	False	False	True	False	False	False	True	True	True
2	bass	False	False	True	False	False	True	True	True	True	False
3	bear	True	False	False	True	False	False	True	True	True	True
4	boar	True	False	False	True	False	False	True	True	True	True

In [15]:
```
training_size = int(len(zoo) * 0.8)
```

In [16]:
```
all_labels = {
    i+1: c for i, c in enumerate(
        [
        'Mammal',
        'Bird',
        'Reptile',
        'Fish',
        'Amphibian',
        'Bug',
        'Invertebrate'
        ])
}
```

In [17]:
```
import networkx as nx

# Create a directed knowledge graph
kg = nx.DiGraph()
cols = [c for c in zoo.columns if c != 'animal_name']

def get_triplets(row, col):
    """return triplets for binary and integer elements
```

```
    Format: (s, p, o)
    """
    if col == 'class_type':
        return (
            all_labels[row[col]],
            'is_a',
            row['animal_name'],
        )

    # integer properties:
    if col in ['legs']:
        return (
            row['animal_name'],
            'has_' + col,
            str(row[col]) + '_legs'
        )

    # binary properties:
    if row[col]:
        return (
            row['animal_name'],
            'has',
            str(col)
        )
    else:
        return ()

# Build triplets
triplets = []
for i, row in zoo.iterrows():
    for col in cols:
        if col == 'animal_name':
            continue
        if col == 'class_type' and i > training_size:
            continue
        triplet = get_triplets(row, col)
        if triplet:
            triplets.append(triplet)

# Build the graph
for (s, p, o) in triplets:
    kg.add_node(s)
    kg.add_node(o)
    kg.add_edge(s, o, predicate=p)
```

In [18]:
```
len(triplets)
```

Out[18]: 842

In [19]:
```
triplets[:20]
```

Out[19]:
```
[('aardvark', 'has', 'hair'),
 ('aardvark', 'has', 'milk'),
 ('aardvark', 'has', 'predator'),
 ('aardvark', 'has', 'toothed'),
 ('aardvark', 'has', 'backbone'),
 ('aardvark', 'has', 'breathes'),
 ('aardvark', 'has_legs', '4_legs'),
 ('aardvark', 'has', 'catsize'),
 ('Mammal', 'is_a', 'aardvark'),
 ('antelope', 'has', 'hair'),
 ('antelope', 'has', 'milk'),
 ('antelope', 'has', 'toothed'),
 ('antelope', 'has', 'backbone'),
 ('antelope', 'has', 'breathes'),
 ('antelope', 'has_legs', '4_legs'),
 ('antelope', 'has', 'tail'),
 ('antelope', 'has', 'catsize'),
 ('Mammal', 'is_a', 'antelope'),
 ('bass', 'has', 'eggs'),
 ('bass', 'has', 'aquatic')]
```

In [20]:
```
triplets[-20:]
```

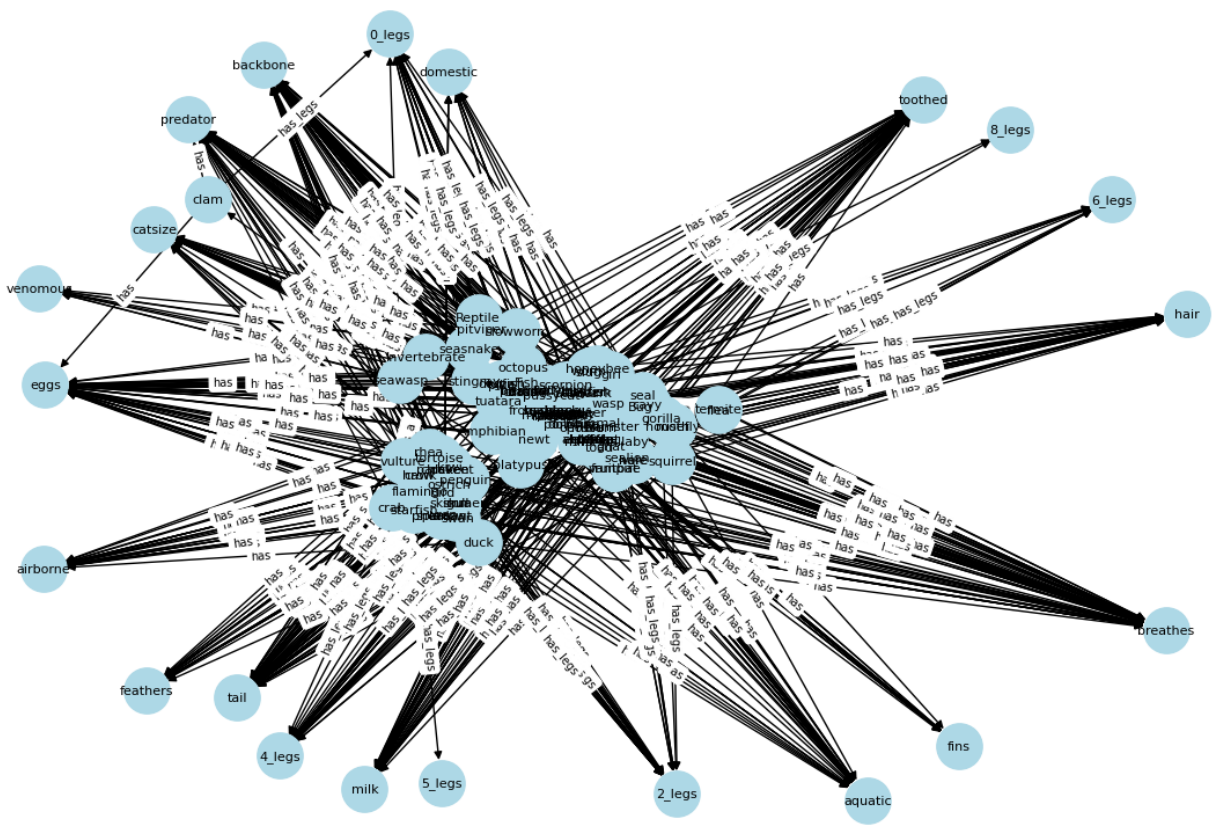
Out[20]:
```
[('wasp', 'has_legs', '6_legs'),
 ('wolf', 'has', 'hair'),
 ('wolf', 'has', 'milk'),
 ('wolf', 'has', 'predator'),
 ('wolf', 'has', 'toothed'),
 ('wolf', 'has', 'backbone'),
 ('wolf', 'has', 'breathes'),
 ('wolf', 'has_legs', '4_legs'),
 ('wolf', 'has', 'tail'),
 ('wolf', 'has', 'catsize'),
 ('worm', 'has', 'eggs'),
 ('worm', 'has', 'breathes'),
 ('worm', 'has_legs', '0_legs'),
 ('wren', 'has', 'feathers'),
 ('wren', 'has', 'eggs'),
 ('wren', 'has', 'airborne'),
 ('wren', 'has', 'backbone'),
 ('wren', 'has', 'breathes'),
 ('wren', 'has_legs', '2_legs'),
 ('wren', 'has', 'tail')]
```

In [21]:
```
import networkx as nx

def create_kg(triplets, label_predicates=None):
    G = nx.DiGraph()
    for (s, p, o) in triplets:
        if label_predicates and p in label_predicates:
            continue
        G.add_node(s)
        G.add_node(o)
        G.add_edge(s, o, predicate=p)
    return G

label_predicates = []

kg = create_kg(triplets, label_predicates=label_predicates)
```

In [22]:
```
kg
```

Out[22]: <networkx.classes.digraph.DiGraph at 0x189dcc8ca50>

In [23]:
```
# kg.visualise()

import matplotlib.pyplot as plt
import networkx as nx

plt.figure(figsize=(12, 8))

pos = nx.spring_layout(kg, seed=42)  # force-directed layout
nx.draw(
    kg,
    pos,
    with_labels=True,
    node_size=800,
    node_color="lightblue",
    font_size=8,
    arrows=True
)

# Draw edge labels (predicates)
edge_labels = nx.get_edge_attributes(kg, 'predicate')
nx.draw_networkx_edge_labels(kg, pos, edge_labels=edge_labels, font_size=7)

plt.show()
```



In [24]:
```
entities = list(zoo.animal_name)
```

In [25]:
```
# from rdf2vec import RDF2VecTransformer
# from rdf2vec.walkers import RandomWalker, CommunityWalker
# import numpy as np
```

```
# # We specify the depth and maximum number of walks per entity
# random_walker = RandomWalker(5, float('inf'))

# transformer = RDF2VecTransformer(vector_size=50, walkers=[random_walker], sg=1)
# # Entities should be a list of URIs that can be found in the KG
# embeddings = np.array(transformer.fit_transform(kg, entities))

# import numpy as np
# from pykeen.triples import TriplesFactory
# from pykeen.pipeline import pipeline

# # Convert list of triples → NumPy array
# triples_array = np.array(triplets, dtype=str)

# # Build TriplesFactory
# tf = TriplesFactory.from_labeled_triples(triples_array)

# # Run PyKEEN
# result = pipeline(
#     training=tf,
#     model='TransE'
# )

# # Extract embeddings
# entity_emb = result.model.get_entity_representations()[0]
# relation_emb = result.model.get_relation_representations()[0]


import numpy as np
from pykeen.triples import TriplesFactory
from pykeen.pipeline import pipeline

# Convert your triples list → NumPy array
triples_array = np.array(triplets, dtype=str)

# Build TriplesFactory
tf = TriplesFactory.from_labeled_triples(triples_array)

# Split into train/test
training, testing = tf.split([0.8, 0.2])

# Run PyKEEN
result = pipeline(
    training=training,
    testing=testing,
    model='TransE'
)

# Extract embeddings
#entity_emb = result.model.get_entity_representations()[0]
#relation_emb = result.model.get_relation_representations()[0]

embeddings = result.model.entity_representations[0]()
relation_emb = result.model.relation_representations[0]()
```

```
using automatically assigned random_state=1073595387
No random seed is specified. Setting to 1939183227.
No cuda devices were available. The model runs on CPU
C:\Users\yathi\anaconda3\Lib\site-packages\torch\utils\data\dataloader.py:668: UserWarning: 'pin_
memory' argument is set as true but no accelerator is found, then device pinned memory won't be u
sed.
  warnings.warn(warn_msg)
```

```
Training epochs on cpu:   0%|          | 0/5 [00:00<?, ?epoch/s]
Training batches on cpu:   0%|          | 0.00/3.00 [00:00<?, ?batch/s]
Training batches on cpu:   0%|          | 0.00/3.00 [00:00<?, ?batch/s]
Training batches on cpu:   0%|          | 0.00/3.00 [00:00<?, ?batch/s]
Training batches on cpu:   0%|          | 0.00/3.00 [00:00<?, ?batch/s]
Training batches on cpu:   0%|          | 0.00/3.00 [00:00<?, ?batch/s]
```

```
WARNING:pykeen.utils:Using automatic batch size on device.type='cpu' can cause unexplained out-o
f-memory crashes. Therefore, we use a conservative small batch_size=32. Performance may be improv
ed by explicitly specifying a larger batch size.
Evaluating on cpu:   0%|          | 0.00/167 [00:00<?, ?triple/s]
```

```
WARNING:torch_max_mem.api:Encountered tensors on device_types={'cpu'} while only ['cuda'] are con
sidered safe for automatic memory utilization maximization. This may lead to undocumented crashes
(but can be safe, too).
INFO:pykeen.evaluation.evaluator:Evaluation took 0.11s seconds
```

In [26]:
```
#embeddings.shape
embeddings.shape
```

Out[26]:
```
torch.Size([128, 50])
```

In [27]:
```
zoo.values[training_size:, :-1]
```

Out[27]:
```
array([['slowworm', False, False, True, False, False, False, True, True,
        True, True, False, False, 0, True, False, False],
       ['slug', False, False, True, False, False, False, False, False,
        False, True, False, False, 0, False, False, False],
       ['sole', False, False, True, False, False, True, False, True,
        True, False, False, True, 0, True, False, False],
       ['sparrow', False, True, True, False, True, False, False, False,
        True, True, False, False, 2, True, False, False],
       ['squirrel', True, False, False, True, False, False, False, True,
        True, True, False, False, 2, True, False, False],
       ['starfish', False, False, True, False, False, True, True, False,
        False, False, False, False, 5, False, False, False],
       ['stingray', False, False, True, False, False, True, True, True,
        True, False, True, True, 0, True, False, True],
       ['swan', False, True, True, False, True, True, False, False, True,
        True, False, False, 2, True, False, True],
       ['termite', False, False, True, False, False, False, False, False,
        False, True, False, False, 6, False, False, False],
       ['toad', False, False, True, False, False, True, False, True,
        True, True, False, False, 4, False, False, False],
       ['tortoise', False, False, True, False, False, False, False,
        False, True, True, False, False, 4, True, False, True],
       ['tuatara', False, False, True, False, False, False, True, True,
        True, True, False, False, 4, True, False, False],
       ['tuna', False, False, True, False, False, True, True, True, True,
        False, False, True, 0, True, False, True],
       ['vampire', True, False, False, True, True, False, False, True,
        True, True, False, False, 2, True, False, False],
       ['vole', True, False, False, True, False, False, False, True,
        True, True, False, False, 4, True, False, False],
       ['vulture', False, True, True, False, True, False, True, False,
        True, True, False, False, 2, True, False, True],
       ['wallaby', True, False, False, True, False, False, False, True,
        True, True, False, False, 2, True, False, True],
       ['wasp', True, False, True, False, True, False, False, False,
        False, True, True, False, 6, False, False, False],
       ['wolf', True, False, False, True, False, False, True, True, True,
        True, False, False, 4, True, False, True],
       ['worm', False, False, True, False, False, False, False, False,
        False, True, False, False, 0, False, False, False],
       ['wren', False, True, True, False, True, False, False, False,
        True, True, False, False, 2, True, False, False]], dtype=object)
```

In [28]:
```
zoo.values[:training_size, 1:-1]
```

Out[28]:
```
array([[True, False, False, ..., False, False, True],
       [True, False, False, ..., True, False, True],
       [False, False, True, ..., True, False, False],
       ...,
       [False, False, True, ..., False, False, False],
       [False, True, True, ..., True, False, False],
       [False, True, True, ..., True, False, False]], dtype=object)
```

In [29]:

```
# from sklearn.svm import SVC
# from sklearn.metrics import (
#     accuracy_score, confusion_matrix
# )

# # Fit a support vector machine on train embeddings and evaluate on test
# clf = SVC(random_state=42)
# clf.fit(embeddings[:training_size, :], zoo.class_type[:training_size])

# test_labels = zoo.class_type[training_size:]
# test_embeddings = embeddings[training_size:, :]
# print(end='Support Vector Machine: Accuracy = ')
# print(accuracy_score(test_labels, clf.predict(test_embeddings)))
# print(confusion_matrix(test_labels, clf.predict(test_embeddings)))


import numpy as np
from pykeen.triples import TriplesFactory
from pykeen.pipeline import pipeline
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

# ---------------------------------------------------------
# 1. Convert your triplets list → NumPy array for PyKEEN
# ---------------------------------------------------------
triples_array = np.array(triplets, dtype=str)

# ---------------------------------------------------------
# 2. Build TriplesFactory
# ---------------------------------------------------------
tf = TriplesFactory.from_labeled_triples(triples_array)

# ---------------------------------------------------------
# 3. Split into train/test for PyKEEN
# ---------------------------------------------------------
training, testing = tf.split([0.8, 0.2])

# ---------------------------------------------------------
# 4. Train TransE model
# ---------------------------------------------------------
result = pipeline(
    training=training,
    testing=testing,
    model='TransE',
    random_seed=42
)

# ---------------------------------------------------------
# 5. Extract embeddings (PyTorch → NumPy)
# ---------------------------------------------------------
raw_emb = result.model.entity_representations[0]().detach().numpy()

# ---------------------------------------------------------
# 6. Reorder embeddings to match zoo DataFrame row order
# ---------------------------------------------------------
entity_to_id = result.training.entity_to_id

ordered_embeddings = np.vstack([
    raw_emb[entity_to_id[name]]
    for name in zoo.animal_name
])
```

```
# ---------------------------------------------------------
# 7. Train/Test split for SVM (aligned with zoo rows)
# ---------------------------------------------------------
train_X = ordered_embeddings[:training_size, :]
train_y = zoo.class_type[:training_size]

test_X = ordered_embeddings[training_size:, :]
test_y = zoo.class_type[training_size:]

# ---------------------------------------------------------
# 8. Train SVM classifier
# ---------------------------------------------------------
clf = SVC(random_state=42)
clf.fit(train_X, train_y)

# ---------------------------------------------------------
# 9. Evaluate
# ---------------------------------------------------------
pred = clf.predict(test_X)

print("Support Vector Machine: Accuracy =", accuracy_score(test_y, pred))
print(confusion_matrix(test_y, pred))
```

```
WARNING:pykeen.utils:using automatically assigned random_state=547064674
INFO:pykeen.triples.splitting:done splitting triples to groups of sizes [545, 167]
WARNING:pykeen.utils:No cuda devices were available. The model runs on CPU
INFO:pykeen.pipeline.api:Using device: None
INFO:pykeen.nn.representation:Inferred unique=False for Embedding()
INFO:pykeen.nn.representation:Inferred unique=False for Embedding()
C:\Users\yathi\anaconda3\Lib\site-packages\torch\utils\data\dataloader.py:668: UserWarning: 'pin_
memory' argument is set as true but no accelerator is found, then device pinned memory won't be u
sed.
  warnings.warn(warn_msg)
Training epochs on cpu:   0%|          | 0/5 [00:00<?, ?epoch/s]
Training batches on cpu:   0%|          | 0.00/3.00 [00:00<?, ?batch/s]
Training batches on cpu:   0%|          | 0.00/3.00 [00:00<?, ?batch/s]
Training batches on cpu:   0%|          | 0.00/3.00 [00:00<?, ?batch/s]
Training batches on cpu:   0%|          | 0.00/3.00 [00:00<?, ?batch/s]
Training batches on cpu:   0%|          | 0.00/3.00 [00:00<?, ?batch/s]
WARNING:pykeen.utils:Using automatic batch size on device.type='cpu' can cause unexplained out-o
f-memory crashes. Therefore, we use a conservative small batch_size=32. Performance may be improv
ed by explicitly specifying a larger batch size.
Evaluating on cpu:   0%|          | 0.00/167 [00:00<?, ?triple/s]
WARNING:torch_max_mem.api:Encountered tensors on device_types={'cpu'} while only ['cuda'] are con
sidered safe for automatic memory utilization maximization. This may lead to undocumented crashes
(but can be safe, too).
INFO:pykeen.evaluation.evaluator:Evaluation took 0.09s seconds
Support Vector Machine: Accuracy = 0.23809523809523808
[[5 0 0 0 0 0 0]
 [4 0 0 0 0 0 0]
 [3 0 0 0 0 0 0]
 [3 0 0 0 0 0 0]
 [1 0 0 0 0 0 0]
 [2 0 0 0 0 0 0]
 [3 0 0 0 0 0 0]]
```

In [30]:

```
# [color_map[i] for i in all_labels[:training_size]]

# 1. Labels aligned with zoo rows
all_labels = list(zoo.class_type)
```

```
# 2. Define color map for your classes
color_map = {
    1: "red",
    2: "green",
    3: "blue",
    4: "orange",
    5: "purple",
    6: "brown",
    7: "pink"
}

# 3. Colors for training samples
colors = [color_map[i] for i in all_labels[:training_size]]
```

In [31]:
```
# baseline
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, accuracy_score


clf = SVC(random_state=42)
clf.fit(zoo.values[:training_size, 1:-1], zoo.class_type[:training_size])

test_labels = zoo.class_type[training_size:]
test_embeddings = zoo.values[training_size:, 1:-1]
print(end='Support Vector Machine: Accuracy = ')
print(accuracy_score(test_labels, clf.predict(test_embeddings)))
print(confusion_matrix(test_labels, clf.predict(test_embeddings)))
```

```
Support Vector Machine: Accuracy = 0.8571428571428571
[[5 0 0 0 0 0 0]
 [0 4 0 0 0 0 0]
 [2 0 0 1 0 0 0]
 [0 0 0 3 0 0 0]
 [0 0 0 0 1 0 0]
 [0 0 0 0 0 2 0]
 [0 0 0 0 0 0 3]]
```

In [32]:
```
embeddings[:training_size, :]
```

Out[32]:
```
tensor([[-0.0275, -0.2548, -0.1056,  ...,  0.0591, -0.1862, -0.2465],
        [-0.2318, -0.1365,  0.0277,  ...,  0.0439, -0.0008, -0.2303],
        [ 0.2207, -0.1500, -0.1967,  ...,  0.2169, -0.2006, -0.0349],
        ...,
        [-0.1198, -0.1106, -0.2197,  ...,  0.0083, -0.1738, -0.0969],
        [ 0.0846,  0.2098,  0.0709,  ...,  0.1923, -0.1940, -0.0083],
        [ 0.2182,  0.1364,  0.1086,  ..., -0.0804, -0.0612,  0.1943]],
       grad_fn=<SliceBackward0>)
```

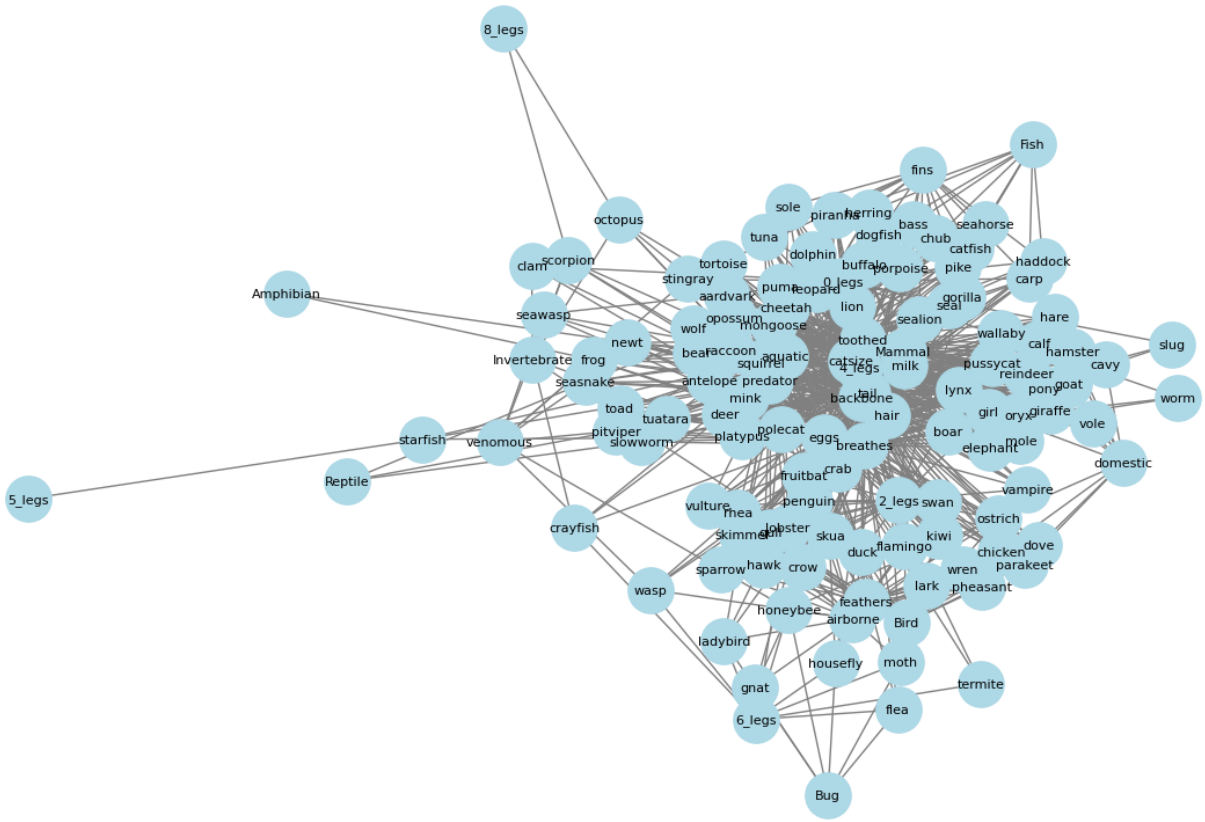
In [33]:
```
import matplotlib.pyplot as plt
import networkx as nx
```

In [34]:
```
nx_graph = nx.Graph()
```

In [35]:
```
nx_graph = nx.Graph()
for (a, p, b) in triplets:
    nx_graph.add_edge(a, b)
```

In [36]:
```
# A = nx.nx_agraph.to_agraph(nx_graph)

import matplotlib.pyplot as plt
```

```python
import networkx as nx

plt.figure(figsize=(12, 8))
pos = nx.spring_layout(nx_graph, seed=42)

nx.draw(
    nx_graph,
    pos,
    with_labels=True,
    node_size=800,
    node_color="lightblue",
    font_size=8,
    edge_color="gray"
)

plt.show()
```



In [37]:
```python
#print(A.string()) # print to screen
```

In [38]:
```python
class Vocabulary:
    """Vocabulary class. Wrapper for dictionaries"""
    label2id = {}
    id2label = {}
    def lookup(self, word):
        """get word id; if not present, insert"""
        if word in self.label2id:
            return self.label2id[word]
        ind = len(self.label2id)
        self.label2id[word] = ind
        return ind

    def inverse_lookup(self, index):
        if len(self.id2label) == 0:
```

```
            self.id2label = {
                ind: label
                for label, ind in self.label2id.items()
            }
        return self.id2label.get(index, None)

vocab = Vocabulary()
nx_graph = nx.Graph()
for (a, p, b) in triplets:
    id1, id2 = vocab.lookup(a), vocab.lookup(b)
    nx_graph.add_edge(id1, id2)
```

In [39]:
```
vocab.inverse_lookup(2)
```

Out[39]:
```
'milk'
```

In [40]:
```
nx_graph.nodes
```

Out[40]:
```
NodeView((0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23,
24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47,
48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71,
72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95,
96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115,
116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127))
```

In [41]:
```
trainamals = [
    vocab.label2id[animal]
    for animal in zoo.animal_name.values[:training_size]
]
```

In [42]:
```
# import networkx as nx
# from karateclub.node_embedding.neighbourhood import Walklets

# model_w = Walklets(dimensions=5)
# model_w.fit(nx_graph)
# embedding = model_w.get_embedding()

import random
from collections import defaultdict

import networkx as nx
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim


class SkipGram(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()
        self.embeddings = nn.Embedding(vocab_size, embed_dim)
        self.output = nn.Linear(embed_dim, vocab_size)

    def forward(self, x):
        emb = self.embeddings(x)
        logits = self.output(emb)
        return logits


class DeepWalkLite:
    def __init__(
```

```python
        self,
        dimensions=64,
        walk_length=40,
        num_walks=10,
        window_size=5,
        lr=0.01,
        epochs=1,
        seed=42,
        device=None,
    ):
        self.dimensions = dimensions
        self.walk_length = walk_length
        self.num_walks = num_walks
        self.window_size = window_size
        self.lr = lr
        self.epochs = epochs
        self.seed = seed
        self.device = device or ("cuda" if torch.cuda.is_available() else "cpu")

        self.model = None
        self.node_to_id = {}
        self.id_to_node = {}

    # ----------------------------
    # Public API
    # ----------------------------
    def fit(self, G: nx.Graph):
        random.seed(self.seed)
        np.random.seed(self.seed)
        torch.manual_seed(self.seed)

        self._build_vocab(G)
        walks = self._generate_walks(G)
        pairs = self._build_training_pairs(walks)

        self._train_skipgram(pairs)

    def get_embedding(self):
        with torch.no_grad():
            emb = self.model.embeddings.weight.detach().cpu().numpy()
        # Return embeddings in node order matching self.id_to_node
        return emb, self.id_to_node

    # ----------------------------
    # Internal helpers
    # ----------------------------
    def _build_vocab(self, G):
        nodes = list(G.nodes())
        self.node_to_id = {node: i for i, node in enumerate(nodes)}
        self.id_to_node = {i: node for node, i in self.node_to_id.items()}

    def _generate_walks(self, G):
        walks = []
        nodes = list(G.nodes())

        for _ in range(self.num_walks):
            random.shuffle(nodes)
            for node in nodes:
                walk = [node]
                current = node
                for _ in range(self.walk_length - 1):
                    neighbors = list(G.neighbors(current))
```

```
                    if not neighbors:
                        break
                    current = random.choice(neighbors)
                    walk.append(current)
                walks.append(walk)
        return walks

    def _build_training_pairs(self, walks):
        pairs = []
        for walk in walks:
            ids = [self.node_to_id[n] for n in walk]
            for i, target in enumerate(ids):
                start = max(0, i - self.window_size)
                end = min(len(ids), i + self.window_size + 1)
                for j in range(start, end):
                    if i == j:
                        continue
                    context = ids[j]
                    pairs.append((target, context))
        return pairs

    def _train_skipgram(self, pairs):
        vocab_size = len(self.node_to_id)
        self.model = SkipGram(vocab_size, self.dimensions).to(self.device)
        optimizer = optim.Adam(self.model.parameters(), lr=self.lr)
        loss_fn = nn.CrossEntropyLoss()

        # simple SGD over pairs
        for epoch in range(self.epochs):
            random.shuffle(pairs)
            total_loss = 0.0
            for target, context in pairs:
                target_t = torch.tensor([target], dtype=torch.long, device=self.device)
                context_t = torch.tensor([context], dtype=torch.long, device=self.device)

                logits = self.model(target_t)
                loss = loss_fn(logits, context_t)

                optimizer.zero_grad()
                loss.backward()
                optimizer.step()

                total_loss += loss.item()

            print(f"Epoch {epoch+1}/{self.epochs} - loss: {total_loss:.4f}")


# ----------------------------
# Example usage
# ----------------------------
if __name__ == "__main__":
    # Example: use your existing nx_graph
    # nx_graph = nx.Graph()
    # nx_graph.add_edge("a", "b")
    # ...

    model = DeepWalkLite(
        dimensions=5,
        walk_length=10,
        num_walks=10,
        window_size=2,
        lr=0.01,
```

```
        epochs=3,
    )
    model.fit(nx_graph)

    embedding, id_to_node = model.get_embedding()
    print("Embedding shape:", embedding.shape)
    # embedding[i] corresponds to node id_to_node[i]
```

```
Epoch 1/3 - loss: 185403.7540
Epoch 2/3 - loss: 183351.4590
Epoch 3/3 - loss: 183292.5221
Embedding shape: (128, 5)
```

In [44]:

```
import random
import numpy as np
import networkx as nx
import torch
import torch.nn as nn
import torch.optim as optim


# ---------------------------------------------------------
# Skip-Gram model
# ---------------------------------------------------------
class SkipGram(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()
        self.embeddings = nn.Embedding(vocab_size, embed_dim)
        self.output = nn.Linear(embed_dim, vocab_size)

    def forward(self, x):
        emb = self.embeddings(x)
        logits = self.output(emb)
        return logits


# ---------------------------------------------------------
# Node2Vec Lite (Python 3.13 compatible)
# ---------------------------------------------------------
class Node2VecLite:
    def __init__(
        self,
        graph: nx.Graph,
        dimensions=64,
        walk_length=30,
        num_walks=10,
        window_size=5,
        p=1.0,
        q=1.0,
        lr=0.01,
        epochs=1,
        seed=42,
        device=None,
    ):
        self.G = graph
        self.dimensions = dimensions
        self.walk_length = walk_length
        self.num_walks = num_walks
        self.window_size = window_size
        self.p = p
        self.q = q
        self.lr = lr
```

```python
        self.epochs = epochs
        self.seed = seed
        self.device = device or ("cuda" if torch.cuda.is_available() else "cpu")

        self.node_to_id = {}
        self.id_to_node = {}
        self.model = None

    # ----------------------------------------------------
    # Public API
    # ----------------------------------------------------
    def fit(self):
        random.seed(self.seed)
        np.random.seed(self.seed)
        torch.manual_seed(self.seed)

        self._build_vocab()
        walks = self._generate_biased_walks()
        pairs = self._build_training_pairs(walks)
        self._train_skipgram(pairs)

    def get_embedding(self):
        with torch.no_grad():
            emb = self.model.embeddings.weight.detach().cpu().numpy()
        return emb, self.id_to_node

    # ----------------------------------------------------
    # Internal helpers
    # ----------------------------------------------------
    def _build_vocab(self):
        nodes = list(self.G.nodes())
        self.node_to_id = {node: i for i, node in enumerate(nodes)}
        self.id_to_node = {i: node for node, i in self.node_to_id.items()}

    def _biased_transition_prob(self, prev, curr, next_node):
        if prev is None:
            return 1.0
        if next_node == prev:
            return 1.0 / self.p
        elif self.G.has_edge(prev, next_node):
            return 1.0
        else:
            return 1.0 / self.q

    def _generate_biased_walks(self):
        walks = []
        nodes = list(self.G.nodes())

        for _ in range(self.num_walks):
            random.shuffle(nodes)
            for start in nodes:
                walk = [start]
                prev = None
                curr = start

                for _ in range(self.walk_length - 1):
                    neighbors = list(self.G.neighbors(curr))
                    if not neighbors:
                        break

                    probs = [
                        self._biased_transition_prob(prev, curr, nxt)
```

```
                    for nxt in neighbors
                ]
                probs = np.array(probs) / np.sum(probs)

                next_node = np.random.choice(neighbors, p=probs)
                walk.append(next_node)
                prev, curr = curr, next_node

            walks.append(walk)

        return walks

    def _build_training_pairs(self, walks):
        pairs = []
        for walk in walks:
            ids = [self.node_to_id[n] for n in walk]
            for i, target in enumerate(ids):
                start = max(0, i - self.window_size)
                end = min(len(ids), i + self.window_size + 1)
                for j in range(start, end):
                    if i != j:
                        pairs.append((target, ids[j]))
        return pairs

    def _train_skipgram(self, pairs):
        vocab_size = len(self.node_to_id)
        self.model = SkipGram(vocab_size, self.dimensions).to(self.device)
        optimizer = optim.Adam(self.model.parameters(), lr=self.lr)
        loss_fn = nn.CrossEntropyLoss()

        for epoch in range(self.epochs):
            random.shuffle(pairs)
            total_loss = 0.0

            for target, context in pairs:
                target_t = torch.tensor([target], dtype=torch.long, device=self.device)
                context_t = torch.tensor([context], dtype=torch.long, device=self.device)

                logits = self.model(target_t)
                loss = loss_fn(logits, context_t)

                optimizer.zero_grad()
                loss.backward()
                optimizer.step()

                total_loss += loss.item()

            print(f"Epoch {epoch+1}/{self.epochs} - loss: {total_loss:.4f}")


# ---------------------------------------------------------
# Example usage
# ---------------------------------------------------------
# model = Node2VecLite(nx_graph, dimensions=9, walk_length=30, num_walks=200)
# model.fit()
# embedding, id_to_node = model.get_embedding()
# print("Embedding shape:", embedding.shape)
```

In [60]:

```
import networkx as nx

# Create an empty graph
```

```
nx_graph = nx.Graph()

# Build triples from zoo dataset
triples = []

feature_cols = [col for col in zoo.columns if col not in ["animal_name", "class_type"]]

for _, row in zoo.iterrows():
    animal = row["animal_name"]
    for feat in feature_cols:
        value = row[feat]
        triples.append((animal, feat, str(value)))


# Add all animals as nodes
for name in zoo.animal_name:
    nx_graph.add_node(name)

# Add edges based on your triples (h, r, t)
# Make sure triples is a list of (head, relation, tail)
for h, r, t in triples:
    if h in nx_graph.nodes() and t in nx_graph.nodes():
        nx_graph.add_edge(h, t)
```

In [62]:

```
# Build triples from zoo dataset
triples = []

feature_cols = [col for col in zoo.columns if col not in ["animal_name", "class_type"]]

for _, row in zoo.iterrows():
    animal = row["animal_name"]
    for feat in feature_cols:
        value = row[feat]
        triples.append((animal, feat, str(value)))
```

In [66]:

```
import networkx as nx

nx_graph = nx.Graph()

# Add all animals as nodes
for name in zoo.animal_name:
    nx_graph.add_node(name)

# Add edges based on triples
for h, r, t in triples:
    if h in nx_graph.nodes() and t in nx_graph.nodes():
        nx_graph.add_edge(h, t)
```

In [67]:

```
model_n2v = Node2VecLite(
    nx_graph,
    dimensions=9,
    walk_length=10,
    num_walks=20,
    window_size=5,
    p=1.0,
    q=1.0,
    epochs=1
)

model_n2v.fit()
embedding, id_to_node = model_n2v.get_embedding()
```

```
Epoch 1/1 - loss: 0.0000
```

In [68]:
```python
import numpy as np

node_to_id = {v: k for k, v in id_to_node.items()}

aligned_embeddings = np.array([
    embedding[node_to_id[name]] for name in zoo.animal_name
])

print("Aligned embedding shape:", aligned_embeddings.shape)
```

```
Aligned embedding shape: (101, 9)
```

In [69]:
```python
trainimals = list(range(training_size))
testimals = list(range(training_size, len(aligned_embeddings)))

print("Train size:", len(trainimals))
print("Test size:", len(testimals))
```

```
Train size: 80
Test size: 21
```

In [70]:
```python
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

clf = SVC(random_state=42)
clf.fit(aligned_embeddings[trainimals], zoo.class_type[:training_size])

test_labels = zoo.class_type[training_size:]
test_embeddings = aligned_embeddings[testimals]

preds = clf.predict(test_embeddings)

print("Support Vector Machine: Accuracy:", "{:.3f}".format(
    accuracy_score(test_labels, preds)
))

print(confusion_matrix(test_labels, preds))
```

```
Support Vector Machine: Accuracy: 0.238
[[5 0 0 0 0 0 0]
 [4 0 0 0 0 0 0]
 [3 0 0 0 0 0 0]
 [3 0 0 0 0 0 0]
 [1 0 0 0 0 0 0]
 [2 0 0 0 0 0 0]
 [3 0 0 0 0 0 0]]
```

In [71]:
```python
# trainimals = list(range(training_size))
# testimals = list(range(training_size, len(embedding)))

import numpy as np
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

# ---------------------------------------------------------
# 1. ALIGN EMBEDDINGS WITH ZOO ROW ORDER
# ---------------------------------------------------------

# node_to_id maps graph node → embedding row index
node_to_id = {v: k for k, v in id_to_node.items()}
```

```python
# reorder embeddings to match zoo.animal_name order
aligned_embeddings = np.array([
    embedding[node_to_id[name]] for name in zoo.animal_name
])

print("Aligned embedding shape:", aligned_embeddings.shape)

# ---------------------------------------------------------
# 2. DEFINE TRAIN / TEST SPLITS
# ---------------------------------------------------------

trainimals = list(range(training_size))
testimals = list(range(training_size, len(aligned_embeddings)))

print("Train size:", len(trainimals))
print("Test size:", len(testimals))

# ---------------------------------------------------------
# 3. TRAIN SVM ON TRAINING EMBEDDINGS
# ---------------------------------------------------------

clf = SVC(random_state=42)
clf.fit(aligned_embeddings[trainimals], zoo.class_type[:training_size])

# ---------------------------------------------------------
# 4. EVALUATE ON TEST EMBEDDINGS
# ---------------------------------------------------------

test_labels = zoo.class_type[training_size:]
test_embeddings = aligned_embeddings[testimals]

preds = clf.predict(test_embeddings)

print("Support Vector Machine: Accuracy:", "{:.3f}".format(
    accuracy_score(test_labels, preds)
))

print(confusion_matrix(test_labels, preds))
```

```
Aligned embedding shape: (101, 9)
Train size: 80
Test size: 21
Support Vector Machine: Accuracy: 0.238
[[5 0 0 0 0 0 0]
 [4 0 0 0 0 0 0]
 [3 0 0 0 0 0 0]
 [3 0 0 0 0 0 0]
 [1 0 0 0 0 0 0]
 [2 0 0 0 0 0 0]
 [3 0 0 0 0 0 0]]
```

In [72]:

```python
# from karateclub import LabelPropagation

# model_lp = LabelPropagation()
# model_lp.fit(nx_graph)
# cluster_membership = model_lp.get_memberships()
# cluster_membership = np.array([cluster_membership[node] for node in range(len(cluster_membershi

import networkx as nx
import numpy as np

# Run label propagation using NetworkX
```

```
communities = nx.algorithms.community.label_propagation_communities(nx_graph)

# communities is a list of sets of nodes
# We convert it into a node → cluster_id mapping
cluster_membership = {}

for cluster_id, community in enumerate(communities):
    for node in community:
        cluster_membership[node] = cluster_id

# Convert to array in node order
ordered_nodes = list(nx_graph.nodes())
cluster_membership_array = np.array([cluster_membership[node] for node in ordered_nodes])
```

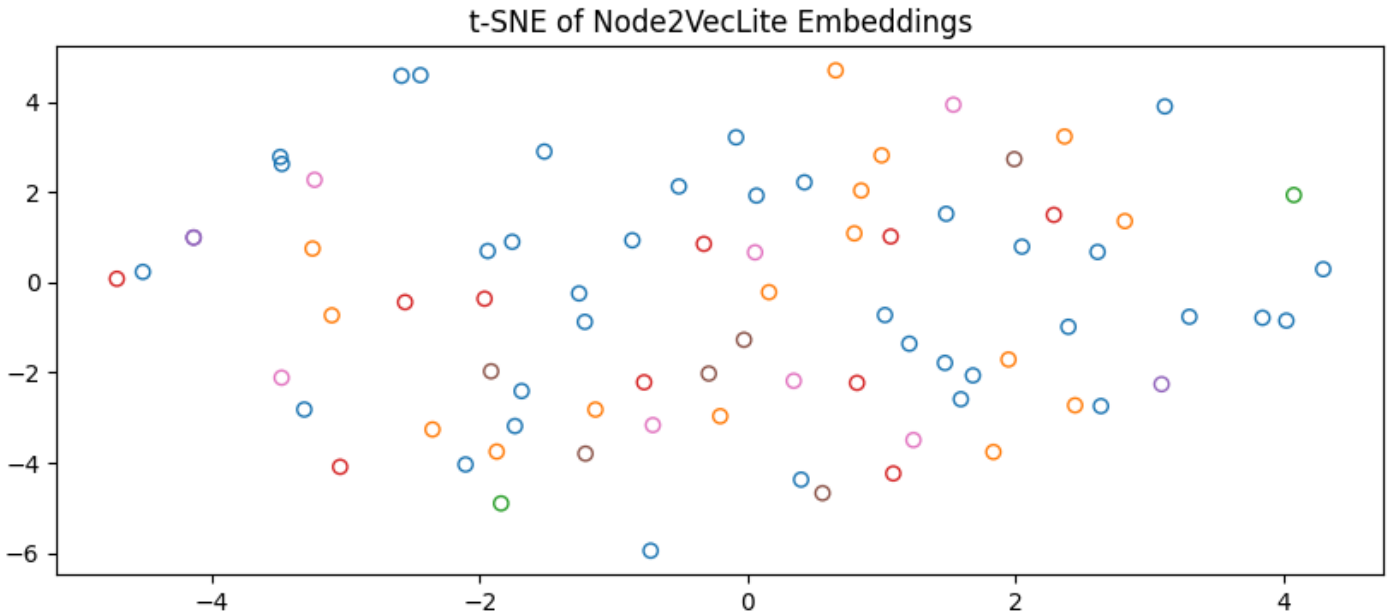
In [73]:
```
cluster_membership_array.reshape(-1, 1).shape
```

Out[73]: (100, 1)

In [80]:
```
# %matplotlib inline
# import matplotlib.pyplot as plt
# from sklearn.manifold import TSNE
# import seaborn as sns
# import itertools

# walk_tsne = TSNE(random_state=42)
# X_walk_tsne = walk_tsne.fit_transform(embedding[trainamals + testimals, :])

# # Define a color map

# palette = itertools.cycle(sns.color_palette())
# color_map = {}
# for i, label in enumerate(set(all_labels)):
#     color_map[label] = next(palette)

# # Plot the train embeddings
# plt.figure(figsize=(10, 4))
# plt.scatter(
#     X_walk_tsne[:training_size, 0],
#     X_walk_tsne[:training_size, 1],
#     edgecolors=[color_map[i] for i in all_labels],
#     facecolors='none',
# )


%matplotlib inline
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
import seaborn as sns
import itertools

walk_tsne = TSNE(random_state=42)

# Use aligned_embeddings, not embedding
X_walk_tsne = walk_tsne.fit_transform(aligned_embeddings[trainimals + testimals, :])

# Define a color map
palette = itertools.cycle(sns.color_palette())
color_map = {label: next(palette) for label in set(all_labels)}

# Plot the train embeddings
plt.figure(figsize=(10, 4))
```

```python
plt.scatter(
    X_walk_tsne[:training_size, 0],
    X_walk_tsne[:training_size, 1],
    edgecolors=[color_map[i] for i in all_labels[:training_size]],
    facecolors='none',
)
plt.title("t-SNE of Node2VecLite Embeddings")
plt.show()
```

```
C:\Users\yathi\anaconda3\Lib\site-packages\joblib\externals\loky\backend\context.py:136: UserWarn
ing: Could not find the number of physical cores for the following reason:
[WinError 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning by setting LOKY_MAX_C
PU_COUNT to the number of cores you want to use.
  warnings.warn(
  File "C:\Users\yathi\anaconda3\Lib\site-packages\joblib\externals\loky\backend\context.py", lin
e 257, in _count_physical_cores
    cpu_info = subprocess.run(
        "wmic CPU Get NumberOfCores /Format:csv".split(),
        capture_output=True,
        text=True,
    )
  File "C:\Users\yathi\anaconda3\Lib\subprocess.py", line 554, in run
    with Popen(*popenargs, **kwargs) as process:
         ~~~~~^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\yathi\anaconda3\Lib\subprocess.py", line 1039, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
    ~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                        pass_fds, cwd, env,
                        ^^^^^^^^^^^^^^^^^^^
    ...<5 lines>...
                        gid, gids, uid, umask,
                        ^^^^^^^^^^^^^^^^^^^^^^
                        start_new_session, process_group)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\yathi\anaconda3\Lib\subprocess.py", line 1554, in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
                       ~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^
                             # no special security
                             ^^^^^^^^^^^^^^^^^^^^^
    ...<4 lines>...
                             cwd,
                             ^^^^
                             startupinfo)
                             ^^^^^^^^^^^^
```



In [77]:

```
import random
import numpy as np
import networkx as nx
import torch
import torch.nn as nn
import torch.optim as optim


# ---------------------------------------------------------
# Skip-Gram model
# ---------------------------------------------------------
class SkipGram(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()
        self.embeddings = nn.Embedding(vocab_size, embed_dim)
        self.output = nn.Linear(embed_dim, vocab_size)

    def forward(self, x):
        emb = self.embeddings(x)
        logits = self.output(emb)
        return logits


# ---------------------------------------------------------
# Node2Vec Lite (Python 3.13 compatible)
# ---------------------------------------------------------
class Node2VecLite:
    def __init__(
        self,
        graph: nx.Graph,
        dimensions=64,
        walk_length=30,
        num_walks=10,
        window_size=5,
        p=1.0,
        q=1.0,
        lr=0.01,
        epochs=1,
        seed=42,
        device=None,
    ):
        self.G = graph
```

```python
        self.dimensions = dimensions
        self.walk_length = walk_length
        self.num_walks = num_walks
        self.window_size = window_size
        self.p = p
        self.q = q
        self.lr = lr
        self.epochs = epochs
        self.seed = seed
        self.device = device or ("cuda" if torch.cuda.is_available() else "cpu")

        self.node_to_id = {}
        self.id_to_node = {}
        self.model = None

    # ------------------------------------------------------
    # Public API
    # ------------------------------------------------------
    def fit(self):
        random.seed(self.seed)
        np.random.seed(self.seed)
        torch.manual_seed(self.seed)

        self._build_vocab()
        walks = self._generate_biased_walks()
        pairs = self._build_training_pairs(walks)
        self._train_skipgram(pairs)

    def get_embedding(self):
        with torch.no_grad():
            emb = self.model.embeddings.weight.detach().cpu().numpy()
        return emb, self.id_to_node

    # ------------------------------------------------------
    # Internal helpers
    # ------------------------------------------------------
    def _build_vocab(self):
        nodes = list(self.G.nodes())
        self.node_to_id = {node: i for i, node in enumerate(nodes)}
        self.id_to_node = {i: node for node, i in self.node_to_id.items()}

    def _biased_transition_prob(self, prev, curr, next_node):
        if prev is None:
            return 1.0
        if next_node == prev:
            return 1.0 / self.p
        elif self.G.has_edge(prev, next_node):
            return 1.0
        else:
            return 1.0 / self.q

    def _generate_biased_walks(self):
        walks = []
        nodes = list(self.G.nodes())

        for _ in range(self.num_walks):
            random.shuffle(nodes)
            for start in nodes:
                walk = [start]
                prev = None
                curr = start
```

```
            for _ in range(self.walk_length - 1):
                neighbors = list(self.G.neighbors(curr))
                if not neighbors:
                    break

                probs = [
                    self._biased_transition_prob(prev, curr, nxt)
                    for nxt in neighbors
                ]
                probs = np.array(probs) / np.sum(probs)

                next_node = np.random.choice(neighbors, p=probs)
                walk.append(next_node)
                prev, curr = curr, next_node

            walks.append(walk)

        return walks

    def _build_training_pairs(self, walks):
        pairs = []
        for walk in walks:
            ids = [self.node_to_id[n] for n in walk]
            for i, target in enumerate(ids):
                start = max(0, i - self.window_size)
                end = min(len(ids), i + self.window_size + 1)
                for j in range(start, end):
                    if i != j:
                        pairs.append((target, ids[j]))
        return pairs

    def _train_skipgram(self, pairs):
        vocab_size = len(self.node_to_id)
        self.model = SkipGram(vocab_size, self.dimensions).to(self.device)
        optimizer = optim.Adam(self.model.parameters(), lr=self.lr)
        loss_fn = nn.CrossEntropyLoss()

        for epoch in range(self.epochs):
            random.shuffle(pairs)
            total_loss = 0.0

            for target, context in pairs:
                target_t = torch.tensor([target], dtype=torch.long, device=self.device)
                context_t = torch.tensor([context], dtype=torch.long, device=self.device)

                logits = self.model(target_t)
                loss = loss_fn(logits, context_t)

                optimizer.zero_grad()
                loss.backward()
                optimizer.step()

                total_loss += loss.item()

            print(f"Epoch {epoch+1}/{self.epochs} - loss: {total_loss:.4f}")


# ----------------------------------------------------------
# Example usage
# ----------------------------------------------------------
# model = Node2VecLite(nx_graph, dimensions=9, walk_length=30, num_walks=200)
# model.fit()
```

```
# embedding, id_to_node = model.get_embedding()
# print("Embedding shape:", embedding.shape)
```

In [78]:
```
# model_n2v = node2vec.fit(window=10, min_count=1, batch_words=4)

model_n2v = Node2VecLite(
    nx_graph,
    dimensions=9,
    walk_length=30,
    num_walks=200,
    window_size=10,   # replaces gensim's window
    p=1.0,
    q=1.0,
    epochs=3
)

model_n2v.fit()
embedding, id_to_node = model_n2v.get_embedding()
```

```
Epoch 1/3 - loss: 0.0000
Epoch 2/3 - loss: 0.0000
Epoch 3/3 - loss: 0.0000
```

In [85]:
```
# model_n2v.wv.get_vector('0')

node_to_id = {v: k for k, v in id_to_node.items()}
vec = embedding[node_to_id['aardvark']]
vec
```

Out[85]:
```
array([1.927, 1.487, 0.901, -2.106, 0.678, -1.235, -0.043, -1.605, -0.752],
      dtype=float32)
```

In [88]:
```
train_embeddings = aligned_embeddings[:training_size]
test_embeddings  = aligned_embeddings[training_size:]
```

In [89]:
```
test_embeddings.shape
```

Out[89]:
```
(21, 9)
```

In [90]:
```
train_embeddings[0]
```

Out[90]:
```
array([1.927, 1.487, 0.901, -2.106, 0.678, -1.235, -0.043, -1.605, -0.752],
      dtype=float32)
```

In [91]:
```
clf = SVC(random_state=42)
clf.fit(
    np.concatenate(
        [
            #zoo.values[:training_size, 1:-1],
            train_embeddings,
            #cluster_membership[trainamals].reshape(-1, 1)
        ],
        axis=1
    ),
    zoo.class_type[:training_size]
)

test_labels = zoo.class_type[training_size:]
test_embeddings = np.concatenate(
    [
        #zoo.values[training_size:, 1:-1],
        test_embeddings,
```

```
        #cluster_membership[testimals].reshape(-1, 1)
    ], axis=1
)
print(end='Support Vector Machine: Accuracy = ')
print(accuracy_score(test_labels, clf.predict(test_embeddings)))
print(confusion_matrix(test_labels, clf.predict(test_embeddings)))
```

```
Support Vector Machine: Accuracy = 0.23809523809523808
[[5 0 0 0 0 0 0]
 [4 0 0 0 0 0 0]
 [3 0 0 0 0 0 0]
 [3 0 0 0 0 0 0]
 [1 0 0 0 0 0 0]
 [2 0 0 0 0 0 0]
 [3 0 0 0 0 0 0]]
```

In [ ]: